

Réversivité

BUT Informatique - S5 - Qualité algorithmique

1 Recherche dichotomique récursive

Question 1

Implémentez la recherche dichotomique de façon récursive.

SOLUTION

```
public static int recherche_dicho(Integer e, Integer [] tab, int bas, int haut){
    if (bas == haut)
        return (tab[bas]==e)?bas:-1;
    else {
        int milieu = (bas + haut) / 2;
        if (e <= tab[milieu])
            return recherche_dicho(e, tab, bas, milieu);
        else
            return recherche_dicho(e, tab, milieu + 1, haut);
    }
}
```

Question 2

Établir l'équation de récurrence pour le coût, en nombre de comparaisons, de cet algorithme et donnez-en la solution.

SOLUTION

Le coût dans le meilleur et dans le pire cas sont identiques car, dans l'algorithme précédent, la recherche n'est pas prématurément interrompue quand la valeur recherchée est `tab[milieu]`.

$$T(n) = \begin{cases} c & \text{si } n = 1 \\ T(n/2) + c' & \text{si } n > 1 \end{cases}$$

où c et c' sont des constantes, et a pour solution $\Theta(\log(n))$. En effet, en supposant que $n = 2^k$ (c'est-à-dire, $k = \log n$)

$$\begin{aligned} T(n) &= T(2^{k-1}) + c' \\ &= T(2^{k-2}) + c' + c \\ &\vdots \\ &= T(2^0) + k * c' \end{aligned}$$

d'où $T(n) = k * c' + c = \log n * c' + c$ d'où $T(n) \in \Theta(\log n)$.

2 Palindrome

Un palindrome est un mot dont les lettres lues de gauche à droite sont les mêmes que celles lues de droite à gauche. Les mots **radar**, **elle**, **été**, **ici** sont des palindromes.

Question 1

Réaliser un prédicat (récursif) qui teste si un mot est un palindrome.

SOLUTION

```
static public boolean palindrome(String mot){
    int lg = mot.length();
    if (lg <= 1) return true;
    if (mot.charAt(0) != mot.charAt(lg-1)) return false;
    else return palindrome(mot.substring(1,lg-1));
}
```

Question 2

Établir l'équation de récurrence pour le coût de votre algorithme et sa solution.

SOLUTION

Dans le meilleur cas, la première et la dernière lettre sont différentes et le coût est $\Theta(1)$. Le coût, dans le pire cas, s'exprime par l'équation de récurrence suivante (où n est la longueur du mot d'entrée):

$$T(n) = \begin{cases} c & \text{si } n \leq 1 \\ T(n-2) + c' & \text{si } n > 1 \end{cases}$$

qui a pour solution $T(n) = \lfloor n/2 \rfloor * c' + c$ d'où $T(n) \in \Theta(n)$.

3 Les tours de Hanoï

Une légende raconte qu'on trouve à Hanoï un temple bouddhiste très ancien, dans lequel se dressent trois mâts d'airain. Sur l'un de ces mâts sont enfilés soixante-quatre disques de jade formant une pyramide.

Les bonzes qui s'affairent dans le temple doivent déplacer tous les disques du premier mât vers celui du milieu, en obéissant à des règles simples:

- seul un disque du sommet d'une pile peut être déplacé;
- on ne peut déplacer qu'un disque à la fois;
- un disque ne peut être posé que sur un disque plus grand ou dans une pile vide;
- à tout moment, on peut déplacer un disque de n'importe quelle pile vers n'importe quelle autre pourvu qu'on respecte toutes les règles ci-dessus.

Lorsqu'ils auront terminé, dit la légende, ce sera la fin du monde.

Question 1

Pour résoudre automatiquement le problème des Tours de Hanoï pour n disques, il suffit de savoir résoudre le problème pour $n-1$ disques. En effet, supposons qu'on soit parti d'une situation où quatre disques sont empilés sur le premier mât et qu'on sache déplacer des piles de 3 disques (ou moins) d'un mât (quelconque) à un autre, comment utiliseriez-vous cette compétence pour déplacer les 4 disques ?

1. déplacer les trois plus petits disques sur le deuxième mât,
2. déplacer le plus grand est sur le troisième mât, et
3. déplacer les trois plus petits disques du deuxième vers le troisième mât;

Question 2

Écrivez une méthode récursive `resolution` qui déplace n disques (un par un), d'une pile source vers une pile de destination en passant, si besoin, par une pile intermédiaire.

```
static Stack<Integer> [] pile = new Stack[3];

public class HanoiException extends Exception {
    public HanoiException(String msg) {
        super(msg) ;
    }
}

public static void deplacer(int source, int dest, boolean trace) throws HanoiException {
    if (pile[source].empty() || (!pile[dest].empty() && pile[dest].peek() < pile[source].peek()))
        throw new HanoiException("Déplacement " + source + " -> " + dest + " impossible!!");
    pile[dest].push(pile[source].pop());
}

public static void resolution(int nb_disques, int source, int dest, int interm, boolean trace) throws HanoiException {
    if (nb_disques > 0){
        resolution(nb_disques-1,source, interm, dest, trace);
        deplacer(source,dest,trace);
        resolution(nb_disques-1,interm, dest, source, trace);
    }
}
```

Question 3

Soit $T(n)$ le nombre d'opérations (déplacements de disques) nécessaires pour résoudre le problème des Tours de Hanoï lorsqu'il y a n disques. Établissez l'équation de récurrence que doit vérifier $T(n)$ et résolvez-la.

Le jeu avec un disque ne nécessite qu'un seul déplacement. Quand il y a $n > 1$ disques, il faut résoudre 2 fois le problème avec $n - 1$ disques et déplacer un disque. D'où l'équation de récurrence:

$$T(n) = \begin{cases} c & \text{si } n = 1 \\ 2 * T(n - 1) + c' & \text{si } n > 1 \end{cases}$$

qui a pour solution $T(n) = \Theta(2^n)$. En effet, on peut établir

$$\begin{aligned} T(n) &= 2 * T(n - 1) + c' \\ &= 2^2 * T(n - 2) + 2 * c' + c' \\ &= 2^3 * T(n - 3) + 2^2 * c' + 2 * c' + c' \\ &= \dots \\ &= 2^{n-1} * T(1) + \left(\sum_{i=0}^{n-2} 2^i \right) * c' \end{aligned}$$

On reconnaît une série géométrique de raison 2 qui a donc pour solution $\sum_{i=0}^{n-2} 2^i = \frac{1-2^{n-1}}{1-2} = 2^{n-1} - 1$,

d'où

$$T(n) = 2^{n-1} * c + (2^{n-1} - 1)c' = 2^{n-1}(c + c') - c'$$

et donc $T(n) \in \Theta(2^n)$.